

Two-Step ADMM Denoising with Cached Cholesky Factors

Julius Zhang

June 2024

1 Introduction

Removing noise is a common task in computer vision and computational photography. This note presents an efficient implementation of an image denoising framework based on the alternating direction method of multipliers (ADMM).

2 Problem statement

Given a noisy image, let $\tilde{x}$ be the flattened vector of its pixels. (For simplicity we assume the image is monochromatic; for a colour image, the same procedure can be repeated independently on each colour channel.) We want to recover a vector x corresponding to the denoised image. This can be formulated as the optimisation problem

$$\text{minimise} \quad D(x, \tilde{x}) + \lambda R(x), \tag{1}$$

where $D(x, \tilde{x})$ is a distance that measures how much the denoised image differs from the noisy input, and $R(x)$ is a regulariser promoting smoothness of the denoised image. The parameter λ controls the trade-off between the regulariser and fidelity to the original image.

3 Method

We take the data term $D(x, \tilde{x})$ to be $\frac{1}{2}\|x - \tilde{x}\|_2^2$ and the regulariser $R(x)$ to be $\|Fx\|_1$, where F is the (banded, sparse) matrix that forms differences of adjacent pixels.

The resulting denoising problem is [1]

$$\text{minimise} \quad \frac{1}{2}\|x - \tilde{x}\|_2^2 + \lambda\|Fx\|_1. \tag{2}$$

Introducing an auxiliary variable z , ADMM rewrites this as

$$\text{minimise} \quad \frac{1}{2}\|x - \tilde{x}\|_2^2 + \lambda\|z\|_1 \tag{3}$$

$$\text{subject to} \quad Fx = z, \tag{4}$$

which yields the updates

$$x^{k+1} = \left(I + \rho F^\top F\right)^{-1} \left(\tilde{x} + \rho F^\top (z^k - u^k)\right), \quad (5)$$

$$z^{k+1} = S_{\lambda/\rho}(Fx^{k+1} + u^k), \quad (6)$$

$$u^{k+1} = u^k + Fx^{k+1} - z^{k+1}, \quad (7)$$

where u is the dual variable, S is the soft-thresholding operator, and $\rho > 0$ is a tunable penalty parameter. See Boyd et al. [1] for background.

4 Implementation

We summarise the techniques used for fast performance. A Python prototype is available [2].

4.1 Cholesky factor caching

The x -update requires solving a linear system with coefficient matrix $I + \rho F^\top F$. This matrix depends only on the image dimensions and on ρ , both of which are fixed, so we compute a Cholesky factorisation of $I + \rho F^\top F$ at load time rather than forming an inverse at every iteration. Since $I + \rho F^\top F$ is diagonally banded and sparse (as is F), its Cholesky factor is also diagonally banded and sparse, and the x -update can be performed in linear time with a small constant factor using the cached factor.

4.2 SIMD

The soft-thresholding operator in the z -update,

$$S_\alpha(v) = \begin{cases} v + \alpha & \text{if } v < -\alpha, \\ v - \alpha & \text{if } v > \alpha, \\ 0 & \text{otherwise,} \end{cases}$$

is applied entrywise and is readily accelerated with SIMD.

4.3 Early stopping

We run only two ADMM iterations and skip the dual update on the second step; empirically this is already enough.

5 Evaluation

Figure 1 shows an example of a denoised image. Runtime depends only on the image dimensions, so for a fixed resolution the cost is constant (linear in the number of pixels). The Python prototype [2] denoises a 128×128 image in 43 milliseconds on average, with a standard deviation of less than five percent of the total runtime. That prototype does

not implement SIMD acceleration; if the soft-thresholding step is skipped, the remaining work finishes in 17 milliseconds on average. For comparison, a SIMD-accelerated CUDA soft-thresholding kernel runs in under 100 microseconds on three colour channels. Due to time constraints I did not port the full Python prototype to CUDA, which would speed the pipeline up substantially by using highly optimised sparse-matrix libraries. With an efficient CUDA implementation, the same 128×128 images should take one or two milliseconds.

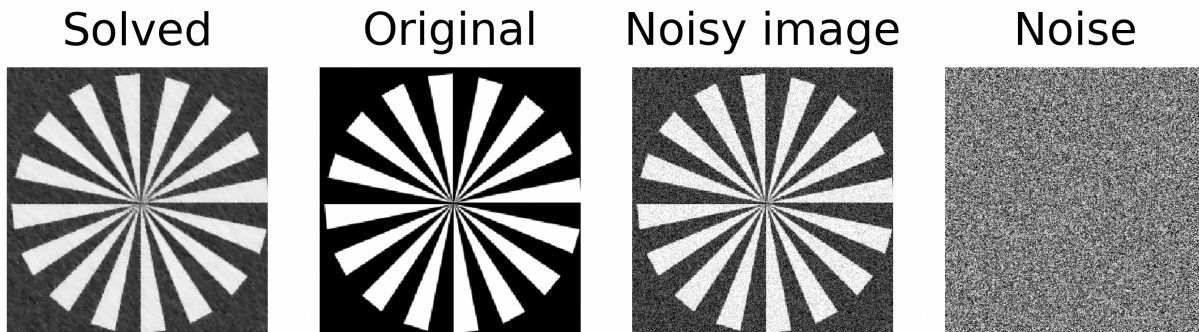


Figure 1: Example of a denoised image produced by the ADMM implementation.

6 Conclusion

This note describes an efficient image denoiser based on ADMM. A natural extension is to handle more heterogeneous noise distributions within the plug-and-play ADMM framework [3]. Another direction is to parallelise the ADMM updates on embedded devices.

References

- [1] Stephen Boyd et al. “Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers”. In: *Found. Trends Mach. Learn.* 3.1 (Jan. 2011), pp. 1–122. ISSN: 1935-8237. DOI: 10.1561/22000000016. URL: <https://doi.org/10.1561/22000000016>.
- [2] Julius Zhang. *A Python prototype of fast ADMM image denoising*. 2024. URL: <https://github.com/JuI3s/cs348k>.
- [3] Rizwan Ahmad et al. “Plug-and-Play Methods for Magnetic Resonance Imaging: Using Denoisers for Image Recovery”. In: *IEEE Signal Processing Magazine* 37.1 (Jan. 2020), pp. 105–116. ISSN: 1558-0792. DOI: 10.1109/msp.2019.2949470. URL: <http://dx.doi.org/10.1109/MSP.2019.2949470>.